

Name: Saiprasad Ashokrao Kawdikar

PRN No: 202501040030

1.1.1. Calculate Momentum

01:31

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

```
mass = float(input())
velocity = float(input())
momentum = mass * velocity
print(f"{momentum:.2f}kgm/s")
```

calculate...

Submit

Explorer

1 mass = float(input())
2 velocity = float(input())
3 momentum = mass * velocity
4 print(f"{momentum:.2f}kgm/s")

Debugger

Average time
0.010 s
9.75 ms

Maximum time
0.012 s
12.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 8 ms Debug

Expected output

Actual output

5.0

5.0

10.0

10.0

50.00kgm/s

50.00kgm/s

Test case 2 9 ms Debug

Expected output

Actual output

2.3

2.3

4.8

4.8

11.04kgm/s

11.04kgm/s

Terminal

Test cases

Name: Saiprasad Ashokrao Kawdikar

PRN No: 202501040030

1.1.2. Conditional Calculation Based on the Number of Digits

03:48

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

$1 \leq \leq 999$

Input Format:

- The input consists of a single integer .

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).

```

n = int(input())
• if 0 < n < 10:
•
print(n**2)
• elif 9 < n < 100:
•
print(f"{n **
(1/2):.2f}")
• elif 99 < n < 1000:
•
print(f"{n **
(1/3):.2f}")
• else:
•
print("Invalid")

```

condition...

Submit

Debugger

Explorer

```
1 n = int(input())
2 if 0 < n < 10:
3     print(n**2)
4 elif 9 < n < 100:
5     print(f"{n ** (1/2):.2f}")
6 elif 99 < n < 1000:
7     print(f"{n ** (1/3):.2f}")
8 else:
9     print("Invalid")
```

Average time
0.005 s
4.57 ms

Maximum time
0.005 s
5.00 ms

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 4 ms Debug

Expected output
9
81

Actual output
9
81

Test case 2 5 ms Debug

Expected output
166
5.50

Actual output
166
5.50

Test case 3 3 ms Debug

Expected output
24
4.90

Actual output
24
4.90

Test case 4 5 ms Debug

Expected output
3456
Invalid

Actual output
3456
Invalid

Terminal Test cases

Prev

Reset

Submit

Next

Name: Saiprasad Ashokrao Kawkdikar

PRN No: 202501040030

1.1.3. Days between Two Dates

05:19

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format .
- The second line contains the second date in the format .

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

```
from datetime import date
date1_input = input().strip()
date2_input = input().strip()
y1, m1, d1 = map(int, date1_input.split('-'))
y2, m2, d2 = map(int, date2_input.split('-'))
date1 = date(y1, m1, d1)
date2 = date(y2, m2, d2)
difference = date2 - date1
print(difference.days)
```

The screenshot shows a code editor with a Python script for calculating the difference in days between two dates. The script is named "noOfDay..." and is being executed. The output shows two test cases passed, with expected and actual outputs matching.

Code:

```
1 from datetime import date
2 date1_input = input().strip()
3 date2_input = input().strip()
4 y1, m1, d1 = map(int, date1_input.split('-'))
5 y2, m2, d2 = map(int, date2_input.split('-'))
6 date1 = date(y1, m1, d1)
7 date2 = date(y2, m2, d2)
8 difference = date2 - date1
9 print(difference.days)
```

Test Results:

- Average time: 0.007 s (7.25 ms)
- Maximum time: 0.010 s (10.00 ms)
- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1 (10 ms):

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

Test case 2 (6 ms):

Expected output	Actual output
2014-07-02	2014-07-02
2014-07-11	2014-07-11
9	9

Navigation: < Prev, Reset, Submit, Next >

Name: Saiprasad Ashokrao Kawdikar

PRN No: 202501040030

1.1.5. Multiplication Table

01:38

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

```
n=int(input())
```


for i in range(1,11):

print(f"{n} x {i} = {n * i}")

The screenshot shows a coding environment with a file explorer on the left, a code editor in the center, and a test results panel at the bottom. The code in the editor is as follows:

```
1 n=int(input())
2 for i in range(1,11):
3     print(f"{n} x {i} = {n * i}")
```

The test results panel displays two test cases, both of which passed. The first test case, labeled 'Test case 1', has an expected output of 8 and an actual output of 8. The second test case, labeled 'Test case 2', has an expected output of 5 and an actual output of 5. The interface also includes a terminal, a submit button, and a debugger.

Test case 1 (4 ms) [Debug](#)

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64
8 x 9 = 72	8 x 9 = 72
8 x 10 = 80	8 x 10 = 80

Test case 2 (4 ms) [Debug](#)

Expected output	Actual output
5	5
5 x 1 = 5	5 x 1 = 5
5 x 2 = 10	5 x 2 = 10
5 x 3 = 15	5 x 3 = 15
5 x 4 = 20	5 x 4 = 20
5 x 5 = 25	5 x 5 = 25
5 x 6 = 30	5 x 6 = 30
5 x 7 = 35	5 x 7 = 35
5 x 8 = 40	5 x 8 = 40
5 x 9 = 45	5 x 9 = 45
5 x 10 = 50	5 x 10 = 50

[Terminal](#) [Test cases](#)

[Prev](#) [Reset](#) [Submit](#) [Next](#)

Name: Saiprasad Ashokrao Kawdikar

PRN No: 202501040030

1.2.1. Pass or Fail

01:11

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer , the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
# Input number of courses
n = int(input())

# Input marks
marks = list(map(int, input().split()))

# Check if any subject is failed
if any(mark < 40 for mark in marks):
    print("Fail")
else:
    # Calculate aggregate percentage
    aggregate = sum(marks) / n

    # Print aggregate percentage formatted to two decimal places
    print(f"Aggregate Percentage: {aggregate:.2f}")

# Determine grade
if aggregate > 75:
    grade = "Distinction"
elif 60 <= aggregate < 75:
    grade = "First Division"
elif 50 <= aggregate < 60:
    grade = "Second Division"
elif 40 <= aggregate < 50:
    grade = "Third Division"
```

```
print(f"Grade: {grade}")
```

Average time
0.006 s
6.20 ms

Maximum time
0.017 s
17.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 17 ms Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2 7 ms Debug

Expected output

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

Actual output

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

Name: Saiprasad Ashokrao Kawdikar

PRN No: 202501040030

1.2.2. Fibonacci Series

32:17

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

```

def fibonacci(n):
    if n==1:
        return 0

    elif n==2:
        return 1

    else:
        return fibonacci(n-1) + fibonacci(n- 2)

n = int(input())
for i in range(1, n + 1):
    print(fibonacci(i), end=" ")

```

Average time: 0.011 s (10.75 ms) | Maximum time: 0.028 s (28.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (4 ms) [Debug] [Test Case]

Expected output	Actual output
5	5
0 · 1 · 1 · 2 · 3 ·	0 · 1 · 1 · 2 · 3 ·

Test case 2 (3 ms) [Debug] [Test Case]

Expected output	Actual output
15	15
0 · 1 · 1 · 2 · 3 · 5 · 8 · 13 · 21 · 34 · 55 · 89 · 144 · 233 · 377 ·	0 · 1 · 1 · 2 · 3 · 5 · 8 · 13 · 21 · 34 · 55 · 89 · 144 · 233 · 377 ·

Terminal | Test cases

< Prev | Reset | Submit | Next >

Name: Saiprasad Ashokrao Kawdikar

PRN No: 202501040030

1.2.3. Pattern - 1

00:50

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

```

n = int(input())
for i in range(1, n + 1):
    for j in range(i):
        print("*", end=" ")
    print()

```

rightangl...

Submit

Explorer

1 n = int(input())
2 for i in range(1, n + 1):
3 for j in range(i):
4 print("*", end=" ")
5 print()

Debugger

Average time

0.006 s
6.00 ms

Maximum time

0.008 s
8.00 ms

2 out of 2 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 7 ms Debug

Expected output

5

*
* *
* * *
* * * *
* * * * *

Actual output

5

*
* *
* * *
* * * *
* * * * *

Test case 2 6 ms Debug

Expected output

4

*
* *
* * *
* * * *

Actual output

4

*
* *
* * *
* * * *

Terminal

Test cases

< Prev

Reset

Submit

Next >

Name: Saiprasad Ashokrao Kawdikar

PRN No: 202501040030

1.2.4. Pattern - 2

01:10

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

```

n = int(input())
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()

```

numberP...

Submit

1

n = int(input())

2

for i in range(1, n + 1):

3

for j in range(1, i + 1):

4

print(j, end=" ")

5

print()

Average time

0.004 s

3.75 ms

Maximum time

0.005 s

5.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

4 ms

Debug

Expected output

Actual output

5

5

1 ·

1 ·

1 · 2 ·

1 · 2 ·

1 · 2 · 3 ·

1 · 2 · 3 ·

1 · 2 · 3 · 4 ·

1 · 2 · 3 · 4 ·

1 · 2 · 3 · 4 · 5 ·

1 · 2 · 3 · 4 · 5 ·

Test case 2

3 ms

Debug

Expected output

Actual output

8

8

1 ·

1 ·

1 · 2 ·

1 · 2 ·

1 · 2 · 3 ·

1 · 2 · 3 ·

1 · 2 · 3 · 4 ·

1 · 2 · 3 · 4 ·

1 · 2 · 3 · 4 · 5 ·

1 · 2 · 3 · 4 · 5 ·

1 · 2 · 3 · 4 · 5 · 6 ·

1 · 2 · 3 · 4 · 5 · 6 ·

1 · 2 · 3 · 4 · 5 · 6 · 7 ·

1 · 2 · 3 · 4 · 5 · 6 · 7 ·

1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 ·

1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 ·

Terminal

Test cases

< Prev

Reset

Submit

Next >